七、Qt 相关（刘志鑫老师）（加粗部分着重记忆，其余内容了解留下印象）

7.1 请描述 Qt 中的内存管理机制，举例说明父对象在对象回收中起到的作用及其原理。

QT 的程序中经常会看到只有 new 而不 delete 的情况，其实是因为 QT 有一套回收内存的机制。这套内存回收机制有以下几个特点：

1）**所有继承于 QObject 类的类，如果在 new 的时候指定了父亲对象，那么父亲对象被清理的时候，这个子对象也会被清理；**

2）内存管理机制主要是依赖于**对象树**和**智能指针**来实现的；

对象树：进行内存管理的对象可以构成一个树状的层次结构，管理其他对象的父对象作为树的父节点，被管理的对象作为子节点，这些类都继承于 QObject 类。而 QObject 类中有私有成员变量 QList<QObject *> children 和 QObject * parent，前者专门存储这个对象的子孙后代们，即它所要管理的那些对象们的地址，后者则记录这个对象的父对象地址。

当一个对象指定一个父对象时，它会把自己添加进父对象的 children 列表中。由此可见一个对象只能指定唯一父对象，但是可以管理很多个其他对象。

Qt 提供了 QSharedPointer、QWeakPointer 和 QScopedPointer 等智能指针来管理动态分配的对象，当一个对象被释放时，其在析构函数中会先释放掉 QList<QObject *> children，即释放它管理的所有子对象，因此，**进行内存管理的对象不必自己 delete，管理它的父对象在 delete 时会先将它释放掉，从而避免内存泄漏等问题**。

7.2 简述 Qt 中的事件机制及其原理

Qt 中事件的来源有两个：

1. **在与用户交互时发生**，比如按下鼠标（mousePressEvent），敲击键盘（keyPressEvent）等。
2. **系统自动发生**，比如计时器事件（timerEvent）等。

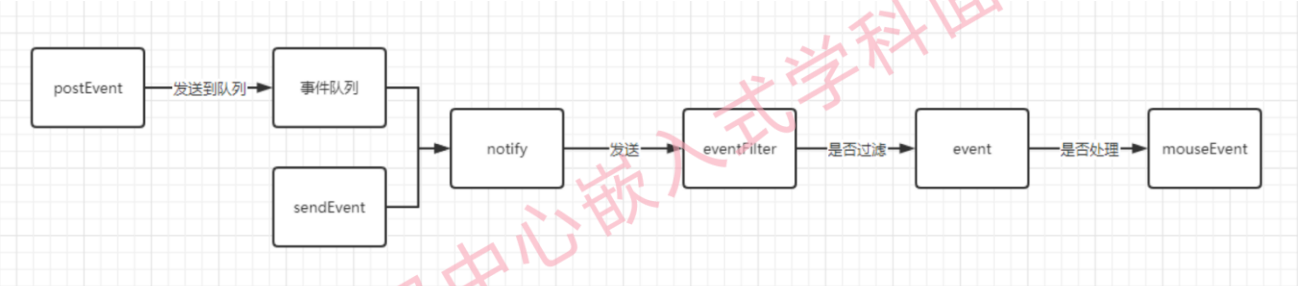
事件由 QObject 类来接收，是 Qt 对象模型的核心，所有需要处理的事件类都必须继承 QObject。

首先 QCoreApplication::exec()开启了事件循环，一直到 QCoreApplication::exit()被调用才终止，所以说事件循环是伴随着 Qt 程序的整个运行周期，事件被分发到事件队列中，当队列中有事件时会不停的将事件发送给 QObject 对象，队列为空时就阻塞。以下为事件处理顺序：

在发生事件时（以鼠标点击事件为例），就会产生一个 QEvent 对象（这里是 QMouseEvent，为 QEvent

华清远见武汉中心嵌入式学科面试宝典

的子类），这个 QEvent 对象会通过 postEvent 被**添加进事件队列**，然后通过 notify 传给**事件过滤器**进行**过滤操作**（eventFilter），如果当前组件没有安装事件过滤器，则会被 event 函数**发放到相应的 mousePressEvent 函数中处理**（函数名将会对应发生的事件）。



➢ sendEvent：使用 notify() 函数直接将事件发送给接收者，发送事件时不会删除该事件，通常是在栈上面创建事件，它是同步事件。

➢ postEvent：将事件添加到事件队列中，并立即返回。事件必须在堆上分配，因为提交事件队列将获得事件的所有权，并在提交后删除它。在事件发布后访问该事件是不安全的，它是异步事件。

7.3 简述 Qt 的信号与槽机制（主要记加粗的关键字）

Qt 的信号与槽（Signals and Slots）是一种高级的回调机制，它不仅提供了对象之间的通信方式，而且还增强了代码的可读性和维护性。这种机制允许一个对象（发送者，Sender）通过发送（Emitting）一个信号（Signal）来通知另一个对象（接收者，Receiver）发生某个事件，接收者通过槽（Slot）响应这个信号。

Qt 信号槽机制主要利用了**事件循环（Event Loop）**和**元对象系统（Meta-Object System）**的以下功能：

➢ **事件循环（Event Loop）**；

➢ **事件处理**：事件循环负责接收和分发事件。当一个信号被触发时，如果接收槽函数的对象位于不同的线程，事件循环将此信号作为事件处理，确保线程安全。

➢ **跨线程通信**：事件循环支持跨线程的信号和槽调用。当一个信号在一个线程内被发射，而槽在另一个线程中，事件循环将信号槽调用作为事件加入目标线程的事件队列，从而实现线程间的安全通信。

➢ **元对象系统（Meta-Object System）**；（详情请看 7.7）

➢ **信号槽连接**：元对象系统提供了一种机制来动态识别和连接信号和槽。这是通过在运行时解析信号和槽的名字来完成的。

➢ **反射能力**：元对象系统允许程序在运行时查询对象的信息，如类名、继承关系、属性、信号和槽等。这对于实现信号槽机制至关重要，因为它使得动态连接成为可能。

华清远见武汉中心嵌入式学科面试宝典

➢ **运行时类型信息（RTTI）**：元对象系统使用一种特殊的运行时类型信息来支持信号和槽的匹配和验证，确保信号的参数类型与槽的参数类型兼容。

通过结合这些事件循环和元对象系统的功能，Qt 的信号槽机制为开发者提供了一个强大且灵活的工具，用于实现对象间的通信。这种机制不仅支持简单的同步调用，也能够处理跨线程的异步调用，同时保持类型安全和高效的运行时性能。

在 Qt 框架中，信号（Signal）和槽（Slot）构成了一种独特的通信机制，它们在软件开发中的角色可以类比于日常生活中的呼叫与应答，以下为信号与槽的特点：

➢ **信号（Signal）**：信号是由 QObject 或其子类的对象发出的消息。它们用于表示某种状态的改变或重要事件的发生。例如，一个按钮被点击，可能会发出一个 clicked()信号。信号本身不包含任何的功能实现，而是作为一个通知机制存在。在 C++中，它们通常被声明为类的成员函数，必须定义在头文件中，但不需要实现。若是自定义的信号，则使用 emit 进行发射。

➢ **槽（Slot）**：槽是用来响应信号的方法。槽可以是任何可调用的代码段，通常是带有权限的成员函数，必须定义在有文件中，且定义后一定要实现。槽也可以是静态函数、全局函数或者是 Lambda 表达式。当与一个信号相连接时，若该信号被发射，相应的槽就会被调用。槽可以和任何信号连接，只要它们的参数兼容。

信号与槽的连接：在 Qt 中，信号和槽之间的连接通常在程序运行时建立，使用 QObject::connect()函数，connect 中有五个参数分别为：发送者（Sender）、发送的信号函数、接收者（Receiver）、调用的槽函数、连接方式。

信号和槽之间的连接可以是以下几种类型：

➢ **直接连接（Qt::DirectConnection）**：信号发射时，槽函数立即在同一线程中执行。

➢ **队列连接（Qt::QueuedConnection）**：信号发射时，槽函数调用被放入事件队列，在接收者线程的事件循环中执行。

➢ **阻塞连接（Qt::BlockingQueuedConnection）**：这种连接类型与 Qt::QueuedConnection 类似，但发送信号的线程在槽函数执行完毕之前会一直处于阻塞状态。它适用于发送方和接收方位于不同线程的情况，但必须确保两者不在同一线程中，以避免死锁。

➢ **自动连接（Qt::AutoConnection）**：Qt 自动选择直接连接或队列连接，取决于信号发射者和接收者是否在同一线程。

➢ **唯一连接（Qt::UniqueConnection）**：这种连接方式确保信号的发送者和接收者之间只能建立一个连接。如果信号的发送者和接收者之间已经建立了连接，则后续针对该发送者和接收者的任何连接 QObject::connect()将会失败。这有助于管理信号和槽的唯一性。

华清远见武汉中心嵌入式学科面试宝典

➢ **Lambda 表达式连接：**这是一种使用匿名函数的方式连接信号和槽。Lambda 表达式提供了一种简洁、方便的方式来表示和定义函数，适用于需要简单表达式或短小代码段的场景。

信号发射时还可以传递参数，这个时候只需要在定义的信号与槽函数的参数列表中写上相匹配的数据类型参数即可。若是需要传递非内置类型的数据，建议将该数据类型定义到一个头文件当中，若是信号与槽在同一个类当中，则只需添加该头文件并在信号与槽的参数列表中定义自定义数据类型即可。若是信号与槽不在同一个类中定义，则这两个类都需要引用这个定义了自定义数据类型的头文件。

7.4 如果 Qt 中现有的窗口部件无法满足当前的需求，如何解决这种问题？

解决方法有两种：添加嵌入的窗口或独立的窗口：

1.**嵌入的窗口：**在原有的窗口部件中添加能够切换界面、嵌入多个界面类的控件，例如 TableWidget、StackedWidget 等。这样操作仍保持原有的主窗口部件显示，然后通过实现界面切换功能显示出不同的子界面从而达到目的。

2.**独立的窗口：**在工程项目中新建一个窗口类（QWidget、QMainWindow、QDialog），使用组合的方式在原窗口的源文件中创建该窗口类的对象并对其初始化，使用 show 函数将新建窗口显示出来即可。

7.5 Qt 多线程的创建方法

在 Qt 中创建多线程有两种方法：

1.**自定义一个线程类继承 QThread，并重写 QThread::run()，**创建线程类对象并调用 **start()** 开启子线程工作，则 run 函数在子线程中运行。

2.使用 QThread 创建子线程，使用 **QObject::moveToThread()**将当前对象移动到子线程中，则 start 子线程后，被信号触发的当前对象的槽函数将在子线程中执行。注意被移动的当前对象必须是 QObject 对象，但不能是 GUI 对象（比如 QWidget 对象），不能有父对象。

7.6 Qt 中多线程如何在创建子线程时传递参数，子线程在执行过程中如何向其它线程传递执行结果？

在 Qt 中创建子线程是传递参数可以通过以下几种方式实现：

1.**公有成员方法传递参数：**创建一个继承于 QThread 的子类，并在子类中定义一个公有的成员函数，通过该函数来设置传递的参数。

华清远见武汉中心嵌入式学科面试宝典

2.**构造函数传递参数**：在创建新的线程对象时，通过构造函数将参数传递给线程对象。然后在线程内部通过成员变量或者函数参数来使用这些参数。

3.**使用 QThread 的 setProperty 函数**：可以使用 setProperty 函数将自定义的属性附加到 QThread 对象上，然后在线程内部使用 QObject::property 函数来获取这些属性。

4.**使用信号与槽机制**：可以在多线程之间使用信号与槽机制来传递参数。在主线程中定义一个信号，然后在子线程中定义一个槽函数来接收这个信号，并将参数传递给槽函数。

5.**定义全局变量来控制参数**：可在两个线程文件中都引用的头文件中，定义全局的变量参数，创建子线程时，改动全局变量可以达到改变子线程中参数变量的目的。

而**子线程在执行过程向其它线程传递执行结果通常使用信号与槽的机制**。子线程可以通过信号和槽（signals and slots）机制与主线程或其他线程通信和传递数据。子线程可以在执行完任务后发射一个信号，这个信号连接到主线程或其他线程中的一个槽函数，并将执行结果传递过去。当然，使用**全局变量或全局函数**，在其他类或线程中调用的方式也同样可以实现操作。

7.7 Qt 编译和 C++编译有什么区别？（元对象系统）

Qt 开发与传统 C++开发相比具有明显的差异。Qt 的**跨平台性**、**图形界面开发**能力、**信号与槽**机制、丰富的功能和库，以及跨平台调试和部署工具等方面的优势，使其成为构建跨平台应用的新选择。Qt 编译和 C++编译最核心的区别在于 Qt 拥有**元对象系统**，这也是 Qt 的核心特性之一，它为框架提供了高级功能，如信号与槽机制、运行时类型信息和动态属性系统。以下是元对象系统的关键方面：

➢ **Q_OBJECT 宏**

- **注册到元对象系统**：Q_OBJECT 宏必须放在所有继承自 QObject 的类的私有声明部分，它使得该类能够被元对象系统识别。
- **信号与槽注册**：使得类可以定义信号和槽，这是 Qt 对象间通信的基础。
- **属性系统**：支持动态属性的设置和获取，通过 Q_PROPERTY 宏注册的属性可以在运行时被访问和修改。

➢ **元对象系统的核心功能**

- **信号与槽机制**：提供了一种安全的回调机制，允许对象之间无需直接引用就能通信。
- **运行时类型信息**：通过 QMetaObject 类，可以在运行时获取类名、属性、方法等信息，而不需要 C++ RTTI（运行时类型识别）。
- **动态属性**：允许在运行时添加、修改和删除对象的属性。
- **国际化支持**：QObject 的 tr()函数用于字符串的国际化。

华清远见武汉中心嵌入式学科面试宝典

- **动态创建对象：**QMetaObject::newInstance()可以在运行时创建对象实例。
- **类型安全的动态类型转换：**qobject_cast<>提供了基于元对象系统的类型转换，适用于跨动态库的情况。

➢ **元对象编译器（MOC）**

- **编译时处理：**MOC 是一个预处理器，它读取包含 Q_OBJECT 宏的源文件，生成额外的 C++代码，这些代码包含了元对象的实现。
- **生成的代码：**MOC 生成的代码通常被包含回原源文件中，或单独编译后链接，以提供信号与槽的连接逻辑、元对象的实现等。

➢ **反射能力（回调）**

- **访问元对象数据：**Qt 的元对象系统赋予了类一定的反射能力，即在运行时能够访问和修改类的元数据，这在C++标准中是不直接提供的。

➢ **Q_PROPERTY 与元对象**

- **声明类的属性：**使用 Q_PROPERTY 宏注册的成员变量，不仅能够响应信号，还能通过元对象系统进行访问和修改，增强了属性的可访问性和动态性。

综上所述，元对象系统是 Qt 区别于其他 C++框架的关键特性，它简化了复杂的应用程序设计，提高了代码的可维护性和灵活性。Qt 的元对象系统通过 MOC 工具的支持，为 QObject 及其子类提供了丰富的动态特性和对象间通信的能力，是 Qt 框架中不可或缺的一部分。

7.8 Qt 如果一个信号的处理方法一直未被执行，有哪些可能？至少说出 3 种

这个问题仍是在考察 Qt 信号与槽机制这个核心概念。如果一个信号的处理方法（槽函数）未被执行，可能的原因有以下几种：

➢ 信号与槽之间的连接可能未建立成功。

➢ 信号发送的条件可能未满足。

➢ 槽函数可能不存在或者写法有误。

➢ 若是自定义的信号，可能没有正确使用 emit 关键字发射信号。

➢ 如果槽函数执行时间过长，可能导致 UI 线程阻塞，使得槽函数未能及时响应。

华清远见武汉中心嵌入式学科面试宝典

7.9 Qt 如何发送一个 HTTP post 请求并接收消息？

使用 Qt 发送一个 Http 协议的 post 请求并接收消息的方法如下：

- 在 Qt 项目的 pro 工程文件中添加 network 网络模块。
- 创建一个 QNetworkAccessManager 管理网络访问和请求类的对象。
- 创建一个 QNetworkRequest 网络请求类的对象，添加 URL 网址，并设置 HTTP 的头部行。
- 使用网络管理对象发出 post 请求。
- 创建一个 QNetworkReply 网络回复类的对象接收 post 请求的回复结果。
- 接收 post 请求回复的结果时要求开启事件循环等待一个完整的回复包被接受才可以结束，否则接收的回复包不全。

7.10 Qt 中如何显示图片和视频？

在 Qt 中添加图片和视频的方法有如下几种：

- 使用 QPixmap 和 QMovie 类在对应的控件上显示电脑路径中的图片和视频。
- 在 UI 界面中添加 QGraphicsView 视图控件，然后在其中添加一个 QGraphicsScene 场景类的对象，最后将 QGraphicsPixmapItem 图片图元类和 QGraphicsVideoItem 视频图元类添加到场景中显示对应路径的图片和视频即可。

华清远见武汉中心嵌入式学科面试宝典
华清远见武汉中心嵌入式学科面试宝典
华清远见武汉中心嵌入式学科面试宝典